

מערכות הפעלה – תרגיל 3

לתרגיל זה 2 חלקים:

- א. פיתוח רשימה מקושרת הבטוחה לשימוש במקביל.
 - ב. כתיבת מודול גרעין להחלפת קריאות מערכת לבחירת המשתמש.
- בסוף התרגיל מפורטות הוראות ההגשה והנחיות כלליות. יש לקרוא אותן בעיון ולהקפיד על כל ההנחיות!

חלק א

עליכם לממש תור בקובץ C בשם `os3q.c`.

התור ימומש באופן הקלאסי עם הקצאה דינמית, כאשר לכל איבר בתור יש ערך **מטיפוס** `long`. לכל תור יש גודל מקסימלי (כמות מקסימלית של איברים) הנקבע באתחול. בנוסף, התור יתמוך במקביליות כך שכל פעולה תעבוד באופן **בטוח** גם כאשר מספר חוטים ניגשים לתור בו-זמנית. כדי לתמוך במקביליות יש לממש את התור באמצעות **מנעול ומשתני תנאי** (ללא semaphore או פעולות אטומיות).

הרשימה עצמה תוגדר בתור struct בשם QueueOS, ותתמוך בפעולות הבאות:

- **אתחול תור** (`init`): מקבלת גודל מקסימלי ומאתחלת תור חדש. יש להניח שפונקציה זו נקראת לפני כל הפעולות, פעם אחת בלבד, וללא פעולות אחרות במקביל על אותו תור.
- **הריסת תור** (`destroy`): מוחקת תור ע"י שחרור כל ההקצאות הרלוונטיות (כולל האיברים שנשארו בתור). יש להניח שפונקציה זו נקראת אחרי כל הפעולות, פעם אחת בלבד, וללא פעולות אחרות במקביל על אותו תור.
- **הוספה** (`enqueue`): מקבלת ערך ומוסיפה אותו לסוף התור. אם התור בגודל המקסימלי, יש לחסום את החוט עד שמתאפשר להוסיף את הערך.
- **הסרה** (`dequeue`): מסירה את האיבר בראש התור ומחזירה את ערכו. אם התור ריק, יש לחסום את החוט עד שמתאפשר להסיר ערך.
- **גודל תור** (`size`): מחזירה את כמות האיברים בתור.
- **סכום** (`sum`): מחזירה את הסכום של כל האיברים בתור.

קובץ H עם הצהרת התור והפעולות מצורף לתרגיל בקובץ `os3q.h`. אין לשנות את קובץ זה ואין להגיש אותו. ניתן ורצוי להוסיף פונקציות עזר והגדרות נוספות לצורך המימוש, בקובץ הקוד `os3q.c`.

שימו לב:

- אין לממש את הרשימה באמצעות מערך אלא כרשימת מצביעים בלבד (המימוש הקלאסי).
- יש להימנע מהגדרת משתנים גלובליים או סטטיים. על הפתרון לתמוך בהגדרה של מספר תורים, כאשר ניתן להשתמש בכל אחד בנפרד ובמקביל לאחרים. לכל תור יתכן גודל מקסימלי שונה.
- יש להגיש את הקוד **ללא main**. הוספת `main` לקוד תגרום לכישלון הבדיקה ופסילה של כל החלק.
- במימוש התוכנית יש להשתמש במנעולים ומשתני תנאי **בלבד** ולא בפונקציות ספריה של שפת C, מלבד `malloc`, `free`, `printf`, `perror`, `exit`. יש לבדוק את ערכי ההחזר של כל קריאות **המערכת** (כולל `malloc`).

מערכות הפעלה – תרגיל 3

חלק ב

עליכם לממש מודול גרעין בקובץ C בשם `os3mod.c`, אשר מחליף אחת או יותר מקריאות המערכת הבאות: `mkdir`, `chdir`, `close`, `dup`.

המודול מקבל 2 פרמטרים: פרמטר ראשון בשם `syscalls` מסוג `int` מערך המציין את מספרי קריאות המערכת שיש להחליף, פרמטר שני בשם `text` מסוג `מחרוזת`.

מספרי קריאות המערכת נתונים ע"י הקבועים `__NR_xyz` (את `xyz` יש להחליף בשם הקריאה). במידה ואחד או יותר מהמספרים שהתקבלו לא תקין (אינו תואם לאחת הקריאות הנתמכות), או שלא הוזן אף מספר, טעינת המודול תיכשל.

יש להחליף כל קריאה שהתקבלה כפרמטר בקריאת מערכת חדשה אשר מדפיסה הודעה **אחרי** ביצוע הפעולה עצמה (קריאת המערכת המקורית):

- עבור `mkdir`, `chdir` יש לשרשר להודעה את שם התיקיה עליה בוצעה הקריאה (עד 12 תווים).
- עבור `close`, `dup` יש לשרשר להודעה את מספר ה-FD עליו בוצעה הקריאה.

במחיקת המודול יש לדאוג לשחזר את הכל למצב הקודם ולהמתין 4 שניות לפני הסיום.

החתימות של קריאות מערכת אלו בקרנל הן:

```
asmlinkage long chdir(const char __user *pathname)
asmlinkage long mkdir(const char __user *pathname, umode_t mode)
asmlinkage long close(unsigned int fd)
asmlinkage long dup(unsigned int fildes)
```

יחד עם מודול הגרעין יש לצרף גם קובץ `Makefile` מתאים אשר מקמפל אותו, וקובץ סקריפט בשם `os3mod.sh` אשר מקבל 3 ארגומנטים ומבצע את הפעולות הבאות (לפי הסדר):

- מעתיק את קובץ הקוד של המודול יחד עם ה-`Makefile` אל התיקיה הנתונה בארגומנט הראשון.
- מקמפל את המודול וטוען אותו אל מערכת ההפעלה, כאשר ארגומנט 2 הוא רשימת המספרים עבור הפרמטר `syscalls` וארגומנט 3 הוא המחרוזת עבור `text`. יש לטעון את המודול רק אם הקומפילציה עברה בהצלחה.
- מוחק את כל הקבצים הזמניים שנוצרו (`sudo make clean`).
- מוחק את 2 הקבצים שהועתקו אל תיקיית היעד (רק אלו שהועתקו, הקבצים המקוריים לא נמחקים).

יש להניח שמתקבלים 3 ארגומנטים והארגומנטים תקינים, ואת קובץ הסקריפט מריצים עם `sudo`.

שורת הרצה של קובץ הסקריפט לדוגמה:

```
sudo ./os3mod.sh ~/kmod 83,32 "Hello"
```

מערכות הפעלה – תרגיל 3

הוראות הגשה:

- יש להגיש את התרגיל בקובץ tar.gz **יחיד**. שם הקובץ הוא ת"ז בלבד (9 ספרות) וסיומת. לדוגמה, אם הת"ז הוא 123456789 אז שם הקובץ להגשה יהיה **123456789.tar.gz**
- הקובץ יכול את 4 קבצי התרגיל **בלבד**, ללא תיקיות או קבצים נסתרים. את הקובץ יש ליצור ב-VM באמצעות הפקודה הבאה:
tar czf <yourid>.tar.gz os3q.c os3mod.c Makefile os3mod.sh
- מצורף לתרגיל סקריפט פייתון (בשם checksub.py). יש להריץ אותו עם שם קובץ ההגשה כפרמטר, לבדיקת תקינות ההגשה. **לא יתקבלו ערעורים על הגשה לא תקינה.**

הנחיות כלליות:

- ההגשה היא **ביחידים**. כל דמיון בין הגשות **ולו הקטן ביותר** יועבר מיידית לטיפול ועדת המשמעת של המכללה ללא אזהרה או אפשרות ערעור. מקורות אונליין (כולל LLM כמו ChatGPT) שקולים להעתקה. פתרו את התרגיל **באופן עצמאי לחלוטין** והימנעו מבעיות.
- הציון על שאלת הקוד מתמקד בנכונות הקוד (קומפילציה ללא אזהרות, בדיקה וטיפול בשגיאות, ניהול משאבים וזיכרון, וכו') ולא עיצוב הקוד (תיעוד, חלוקה לפונקציות, וכו'). עם זאת, יש לכתוב קוד קריא וברור. הקוד יקומפל באמצעות הפקודה:
gcc <codefiles> -Wall -pthread -o <exefile>
- בכל ערעור על קוד יש לצרף תמונת מסך של הטרימינל עם הפלט של פקודת **hostnamectl**, ביצוע **gcc** לקוד, והרצה של הקוד עם פלט תקין – מתוך VM על גבי **vagrant** כפי שנלמד. מי שלא פועל לפי ההנחיות (מערכת הפעלה שונה, או ללא **vagrant**) – **מוותר על הזכות לערער**.
- **הגשות באיחור** יתקבלו עם הפחתה של 10 נק' לכל יום איחור מעבר למועד ההגשה, אשר מסתיים בדיוק **בשעה 23:00**. דאגו להגיש מספיק זמן מראש, לא יתקבל אישור על פספוס ברגע האחרון. במקרה של מחלה, מילואים, או כל סיבה מוצדקת אחרת, יש לשלוח אישור אל המרצה **לפחות 24 שעות** לפני ההגשה.